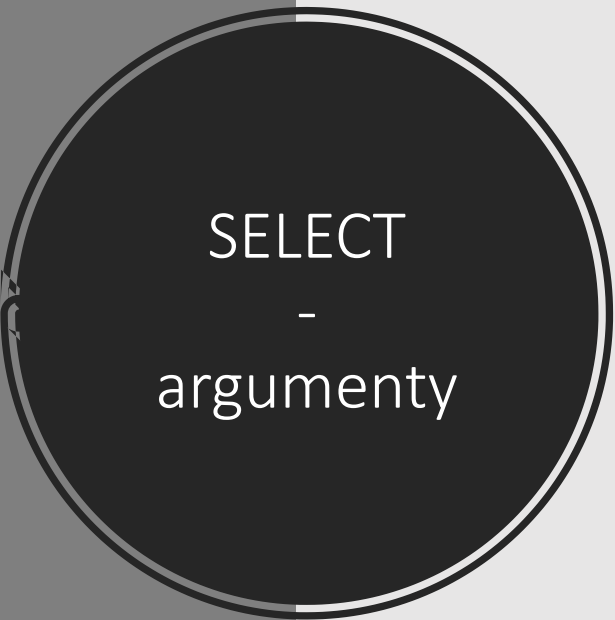


Wstęp do SQL

Baza danych: SQL Server

Przemysław Starosta – 2026



SELECT - argumenty

Określa kolumny zwracane przez zapytanie.

Argumenty

- * - zwraca wszystkie kolumny
- **DISTINCT**
 - Określa, że tylko unikalne wiersze mogą pojawić się w zestawie wyników. Wartości null są uważane za równe
- **TOP (expression) [PERCENT] [WITH TIES]**
 - Wskazuje, że tylko określony pierwszy zestaw lub procent wierszy zostanie zwrócony z zestawu wyników zapytania. wyrażenie może być liczbą lub procentem wierszy.
- <select_list>
 - Kolumny, które mają zostać wybrane do zestawu wyników. Lista wyboru to seria wyrażeń oddzielonych przecinkami. Maksymalna liczba wyrażeń, które można określić na liście wyboru, wynosi 4096.

Proste
zapytania
-
SELECT

Ten kod zwraca wszystkie wiersze (nie określono klauzuli WHERE) i wszystkie kolumny (przy użyciu *)

```
SELECT *  
FROM Employees
```

-- Alternatywnie

```
SELECT e.*  
FROM Employees AS e
```

Proste zapytania

	employee_id	first_name	last_name	email	phone_number	hire_date	job_id	salary	commission_pct	manager_id	department_id
1	100	Steven	King	SKING	1.515.555.0100	2013-06-17	AD_PRES	24000.00	NULL	NULL	90
2	101	Neena	Yang	NYANG	1.515.555.0101	2015-09-21	AD_VP	17000.00	NULL	100	90
3	102	Lex	Garcia	LGARCIA	1.515.555.0102	2011-01-13	AD_VP	17000.00	NULL	100	90
4	103	Alexander	James	AJAMES	1.590.555.0103	2016-01-03	IT_PROG	9000.00	NULL	102	60
5	104	Bruce	Miller	BMILLER	1.590.555.0104	2017-05-21	IT_PROG	6000.00	NULL	103	60
6	105	David	Williams	DWILLIAMS	1.590.555.0105	2015-06-25	IT_PROG	4800.00	NULL	103	60
7	106	Valli	Jackson	VJACKSON	1.590.555.0106	2016-02-05	IT_PROG	4800.00	NULL	103	60

select

*

from employees

SELECT - TOP

```
select  
top(5) last_name  
from Employees
```

	last_name
1	King
2	Yang
3	Garcia
4	James
5	Miller

5 rows

```
select  
top(10) percent last_name  
from Employees
```

	last_name
1	King
2	Yang
3	Garcia
4	James
5	Miller
6	Williams
7	Jackson
8	Nguyen
9	Gruenberg
10	Faviet
11	Chen

11 rows

SELECT - TOP WITH TIES

```
select  
top(6) salary  
from Employees  
order by salary
```

	salary
1	2100.00
2	2200.00
3	2200.00
4	2400.00
5	2400.00
6	2500.00

6 rows

WITH TIES - Jeśli ostatnia wartość się powtarza, pokaże wszystkie wiersze z tą wartością

```
select  
top(6) with ties salary  
from Employees  
order by salary
```

	salary
1	2100.00
2	2200.00
3	2200.00
4	2400.00
5	2400.00
6	2500.00
7	2500.00
8	2500.00
9	2500.00
10	2500.00
11	2500.00

11 rows

Wybieranie kolumn

Zwracam wszystkie wiersze (nie określono klauzuli WHERE) i tylko kolumny (first_name, last_name, salary) z tabeli Employees.

```
select  
    first_name,  
    last_name,  
    salary  
from Employees
```

	first_name	last_name	salary
1	Steven	King	24000.00
2	Neena	Yang	17000.00
3	Lex	Garcia	17000.00
4	Alexander	James	9000.00
5	Bruce	Miller	6000.00
6	David	Williams	4800.00
7	Valli	Jackson	4800.00

Aliasy

```
select
    first_name "to jest imię",
    last_name as nazwisko,
    placa = salary,
    salary + 999 z_podwyzka
from Employees
```

	to jest imię	nazwisko	placa	z_podwyzka
1	Steven	King	24000.00	24999.00
2	Neena	Yang	17000.00	17999.00
3	Lex	Garcia	17000.00	17999.00
4	Alexander	James	9000.00	9999.00
5	Bruce	Miller	6000.00	6999.00
6	David	Williams	4800.00	5799.00

Brak aliasów

```
select
    'Imię ',
    first_name,
    'Nazwisko ',
    last_name,
    'Płaca ',
    salary
from Employees
```

	(No column name)	first_name	(No column name)	last_name	(No column name)	salary
1	Imię	Steven	Nazwisko	King	Płaca	24000.00
2	Imię	Neena	Nazwisko	Yang	Płaca	17000.00
3	Imię	Lex	Nazwisko	Garcia	Płaca	17000.00
4	Imię	Alexander	Nazwisko	James	Płaca	9000.00
5	Imię	Bruce	Nazwisko	Miller	Płaca	6000.00
6	Imię	David	Nazwisko	Williams	Płaca	4800.00

WHERE

Zwracam tylko wiersze dla first_name, last_name i salary, które mają salary większe od 15000.

```
select
    first_name,
    last_name,
    salary
from Employees
where salary > 15000
```

	first_name	last_name	salary
1	Steven	King	24000.00
2	Neena	Yang	17000.00
3	Lex	Garcia	17000.00

LIKE(1/3)

Funkcja **LIKE** w SQL służy do wyszukiwania wierszy w tabeli, które pasują do określonego wzorca tekstowego.

Zamiast szukać dokładnego dopasowania (jak przy użyciu operatora =), LIKE pozwala na użycie symboli wieloznacznych (wildcard characters), które reprezentują jeden lub wiele znaków. Dzięki temu możesz znaleźć dane, które zawierają określone fragmenty tekstu, zaczynają się na daną literę, kończą się na określone słowo, itp.

Dwa główne symbole wieloznaczne używane z LIKE to:

- **% (procent): Reprezentuje zero lub więcej dowolnych znaków.**
 - WHERE nazwa_kolumny LIKE 'a%': Znajdzie wszystkie wartości w kolumnie nazwa_kolumny, które zaczynają się na literę "a" (np. "adam", "ania", "arbuz", "a").
 - WHERE nazwa_kolumny LIKE '%tekst%': Znajdzie wszystkie wartości, które zawierają w sobie ciąg znaków "tekst" (np. "jakiś tekst", "tekst na początku", "koniec tekstu").
 - WHERE nazwa_kolumny LIKE 'p%o': Znajdzie wszystkie wartości, które zaczynają się na "p" i kończą na "o" (np. "piwo", "piano", "polo").
- **(podkreślenie): Reprezentuje dokładnie jeden dowolny znak.**
 - WHERE nazwa_kolumny LIKE 'k_t': Znajdzie wszystkie wartości, które zaczynają się na "k", mają jeden dowolny znak w środku i kończą się na "t" (np. "kot", "kit", "kat").
 - WHERE nazwa_kolumny LIKE '_a%': Znajdzie wszystkie wartości, które mają dowolny pierwszy znak, po którym następuje "a" i cokolwiek dalej (np. "bartek", "california").

LIKE(2/3)

	employee_id	first_name	last_name	salary	department_id
1	100	Steven	King	24000.00	90
2	116	Shelli	Baida	2900.00	30
3	117	Sigal	Tobias	2800.00	30
4	123	Shanta	Vollman	6500.00	50
5	128	Steven	Markle	2200.00	50
6	138	Stephen	Stiles	3200.00	50

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
from employees
where
    first_name like 'S%'
```

LIKE(3/3)

	employee_id	first_name	last_name	salary	department_id
1	105	David	Williams	4800.00	60
2	106	Valli	Jackson	4800.00	60
3	108	Nancy	Gruenberg	12008.00	100
4	109	Daniel	Faviet	9000.00	100
5	119	Karen	Colmenares	2500.00	30
6	120	Matthew	Weiss	8000.00	50
7	122	Payam	Kaufling	7900.00	50

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
from employees
where
    first_name like '_a%'
```

BETWEEN

Operator BETWEEN w SQL służy do filtrowania wierszy na podstawie zakresu wartości dla określonej kolumny. Pozwala sprawdzić, czy wartość w danej kolumnie mieści się w podanym zamkniętym przedziale, czyli łącznie z wartościami granicznymi.

Jest to wygodniejszy i bardziej czytelny sposób na sprawdzanie zakresów niż używanie kombinacji operatorów porównania ($\geq$ i $\leq$).

	employee_id	first_name	last_name	salary	department_id
1	104	Bruce	Miller	6000.00	60
2	105	David	Williams	4800.00	60
3	106	Valli	Jackson	4800.00	60
4	111	Ismael	Sciarra	7700.00	100
5	112	Jose Manuel	Urman	7800.00	100
6	113	Luis	Popp	6900.00	100

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
from employees
where
    salary between 4800 and 8000
```



AND, OR

	employee_id	first_name	last_name	salary	commission_pct	department_id
1	100	Steven	King	24000.00	NULL	90
2	101	Neena	Yang	17000.00	NULL	90
3	102	Lex	Garcia	17000.00	NULL	90
4	103	Alexander	James	9000.00	NULL	60
5	104	Bruce	Miller	6000.00	NULL	60
6	108	Nancy	Gruenberg	12008.00	NULL	100
7	109	Daniel	Faviet	9000.00	NULL	100

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    commission_pct,
    department_id
from employees
where
    salary > 5000
    and department_id <> 30
    or department_id = 50
```



IN

Operator IN w SQL służy do sprawdzania, czy wartość w określonej kolumnie pasuje do którejkolwiek z wartości wymienionych na liście.

Jest to skrócony i bardziej czytelny sposób na łączenie wielu warunków OR z porównaniami równości (=).

Zamiast pisać wiele warunków typu kolumna = wartość1 OR kolumna = wartość2 OR kolumna = wartość3, możesz użyć operatora IN z listą tych wartości.

	employee_id	first_name	last_name	salary	department_id
1	114	Den	Li	11000.00	30
2	115	Alexander	Khoo	3100.00	30
3	116	Shelli	Baida	2900.00	30
4	117	Sigal	Tobias	2800.00	30
5	118	Guy	Himuro	2600.00	30
6	119	Karen	Colmenares	2500.00	30
7	200	Jennifer	Whalen	4400.00	10
8	201	Michael	Martinez	13000.00	20
9	202	Pat	Davis	6000.00	20

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    department_id
from employees
where
    department_id IN (10,20,30)
```



NULL

Wartość NULL w SQL reprezentuje brakującą, nieznaną, nieokreśloną lub nieaplikowalną wartość danych w kolumnie tabeli.

Nie jest to zero, nie jest to pusty ciąg znaków ("), a nawet nie jest to spacja (' ,).

NULL to unikalny znacznik, który wskazuje na nieistnienie wartości.

Możesz sobie wyobrazić, że pusta komórka w arkuszu kalkulacyjnym niekoniecznie oznacza zero. Może oznaczać, że dane nie zostały jeszcze wprowadzone, są nieznane lub po prostu nie mają zastosowania w danym kontekście.

W SQL NULL pełni podobną rolę.

Kluczowe cechy i konsekwencje używania wartości NULL:

- **Nie jest wartością:** Kluczowe jest zrozumienie, że NULL nie jest wartością w tradycyjnym sensie. Dlatego nie można go porównywać za pomocą standardowych operatorów porównania (=, <>, >, <). Próba porównania z NULL zawsze da w wyniku UNKNOWN.
- **Aby sprawdzić, czy wartość w kolumnie jest NULL, używa się specjalnych operatorów:**
 - **IS NULL:** Sprawdza, czy wartość jest NULL.
 - **IS NOT NULL:** Sprawdza, czy wartość nie jest NULL.



IS NULL

	employee_id	first_name	last_name	salary	commission_pct	department_id
1	100	Steven	King	24000.00	NULL	90
2	101	Neena	Yang	17000.00	NULL	90
3	102	Lex	Garcia	17000.00	NULL	90
4	103	Alexander	James	9000.00	NULL	60
5	104	Bruce	Miller	6000.00	NULL	60
6	105	David	Williams	4800.00	NULL	60
7	106	Valli	Jackson	4800.00	NULL	60
8	107	Diana	Nguyen	4200.00	NULL	60

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    commission_pct,
    department_id
from employees
where
    commission_pct is null
```

ISNULL

Funkcja ISNULL w SQL Server służy do zastępowania wartości NULL określoną wartością zastępczą.

Jest to podobne w działaniu do funkcji COALESCE, ale przyjmuje dokładnie dwa argumenty. Jeśli pierwszy argument jest NULL, funkcja zwraca drugi argument. Jeśli pierwszy argument nie jest NULL, funkcja zwraca jego wartość bez zmian.

	last_name	salary	commission_pct	pelna_pensja
43	Davies	3100.00	NULL	3100.00
44	Matos	2600.00	NULL	2600.00
45	Vargas	2500.00	NULL	2500.00
46	Singh	14000.00	0.40	14000.40
47	Partners	13500.00	0.30	13500.30
48	Errazuriz	12000.00	0.30	12000.30

```
select
    last_name,
    salary,
    commission_pct,
    salary + ISNULL(commission_pct,0) pelna_pensja
from employees
```



COALESCE (1/2)

Funkcja COALESCE w SQL Server (i w wielu innych systemach baz danych SQL) służy do zwracania pierwszej nie-NULLowej wartości z listy podanych argumentów.

Przyjmuje ona zmienną liczbę argumentów i przetwarza je w kolejności, aż znajdzie wartość, która nie jest NULL.

Jak tylko taka wartość zostanie znaleziona, funkcja ją zwraca i przestaje przetwarzać pozostałe argumenty. Jeśli wszystkie podane argumenty są NULL, funkcja COALESCE również zwraca NULL.

COALESCE (2/2)

```
create table coalesce_problem (col1 int, col2 int, col3 int);
```

```
insert into coalesce_problem values (null,null,1);
```

```
insert into coalesce_problem values (null,2,1);
```

```
insert into coalesce_problem values (3,2,1);
```

```
select * from coalesce_problem
```

	col1	col2	col3
1	NULL	NULL	1
2	NULL	2	1
3	3	2	1

```
select
```

```
    col1,
```

```
    col2,
```

```
    col3,
```

```
    coalesce(col1,col2,col3) result
```

```
from coalesce_problem
```

	col1	col2	col3	result
1	NULL	NULL	1	1
2	NULL	2	1	2
3	3	2	1	3



NEGACJE

IS NOT NULL
NOT BETWEEN
NOT IN
NOT LIKE



HIERARCHIA OPERATORÓW

W warunkach **ZAWSZE** pierwszy sprawdzany jest **AND** potem **OR**.

Można zmienić tę kolejność poprzez zastosowanie **()**

Obsługa NULL(1/3)

	employee_id	first_name	last_name	salary	commission_pct	result
1	100	Steven	King	24000.00	NULL	NULL
2	101	Neena	Yang	17000.00	NULL	NULL
3	102	Lex	Garcia	17000.00	NULL	NULL
4	103	Alexander	James	9000.00	NULL	NULL
5	104	Bruce	Miller	6000.00	NULL	NULL
6	105	David	Williams	4800.00	NULL	NULL
7	106	Valli	Jackson	4800.00	NULL	NULL

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    commission_pct,
    salary + commission_pct result
from employees
```

Obsługa NULL(2/3)

	employee_id	first_name	last_name	salary	commission_pct	result
1	100	Steven	King	24000.00	NULL	24000.00
2	101	Neena	Yang	17000.00	NULL	17000.00
3	102	Lex	Garcia	17000.00	NULL	17000.00
4	103	Alexander	James	9000.00	NULL	9000.00
5	104	Bruce	Miller	6000.00	NULL	6000.00
6	105	David	Williams	4800.00	NULL	4800.00

```
select
    employee_id,
    first_name,
    last_name,
    salary,
    commission_pct,
    salary + ISNULL(commission_pct,0) result
from employees
```

Obsługa NULL(3/3)

	col1	col2	col3
1	NULL	NULL	1
2	NULL	2	1
3	3	2	1

```
select * from coalesce_problem
```

	col1	col2	col3	result
1	NULL	NULL	1	1
2	NULL	2	1	2
3	3	2	1	3

```
select
    col1,
    col2,
    col3,
    coalesce(col1,col2,col3) result
from coalesce_problem
```

ORDER BY (1/4)

	employee_id	first_name	last_name	salary
1	121	Adam	Fripp	8200.00
2	196	Alana	Walsh	3100.00
3	147	Alberto	Errazuriz	12000.00
4	103	Alexander	James	9000.00
5	115	Alexander	Khoo	3100.00
6	185	Alexis	Bull	4100.00
7	158	Allan	McEwen	9000.00

```
select
    employee_id,
    first_name,
    last_name,
    salary
from employees
order by first_name
```

ORDER BY (2/4)

	employee_id	first_name	last_name	salary
1	100	Steven	King	24000.00
2	101	Neena	Yang	17000.00
3	102	Lex	Garcia	17000.00
4	145	John	Singh	14000.00
5	146	Karen	Partners	13500.00
6	201	Michael	Martinez	13000.00

```
select
    employee_id,
    first_name,
    last_name,
    salary
from employees
order by salary DESC
```

ORDER BY (3/4)

	employee_id	first_name	last_name	department_id	salary
1	178	Kimberely	Grant	NULL	7000.00
2	200	Jennifer	Whalen	10	4400.00
3	201	Michael	Martinez	20	13000.00
4	202	Pat	Davis	20	6000.00
5	114	Den	Li	30	11000.00
6	115	Alexander	Khoo	30	3100.00

```
select
    employee_id,
    first_name,
    last_name,
    department_id,
    salary
from employees
order by department_id, salary DESC
```

ORDER BY (4/4)

	employee_id	first_name	last_name	department_id	salary
1	100	Steven	King	90	24000.00
2	101	Neena	Yang	90	17000.00
3	102	Lex	Garcia	90	17000.00
4	103	Alexander	James	60	9000.00
5	104	Bruce	Miller	60	6000.00

```
select
    employee_id,
    first_name,
    last_name,
    department_id,
    salary
from employees
order by 1
```

DISTINCT

Poniższy przykład wykorzystuje DISTINCT, aby zapobiec pobieraniu zduplikowanych wierszy.

```
select  
    distinct  
        department_id  
from employees
```

	department_id
1	NULL
2	10
3	20
4	30
5	40
6	50
7	60
8	70
9	80
10	90
11	100
12	110

Kolejność wykonywania klauzul (1/3)

Messages

Msg 207, Level 16, State 1, Line 1
Invalid column name 'result'.

```
select
    employee_id,
    first_name,
    last_name,
    department_id,
    salary + 100 result
from employees
where result > 10000
```

Kolejność
wykonywania
klauzul (2/3)

	employee_id	first_name	last_name	department_id	result
1	132	TJ	Olson	50	2200.00
2	128	Steven	Markle	50	2300.00
3	136	Hazel	Philtanker	50	2300.00
4	135	Ki	Gee	50	2500.00
5	127	James	Landry	50	2500.00
6	119	Karen	Colmenares	30	2600.00
7	140	Joshua	Patel	50	2600.00

```
select
    employee_id,
    first_name,
    last_name,
    department_id,
    salary + 100 result
from employees
order by result
```



Kolejność
wykonywania
klauzul (3/3)

- 1 - FROM
- 2 - WHERE
- 3 - GROUP BY
- 4 - HAVING
- 5 - SELECT
- 6 - ORDER BY

SELECT INTO

-
Tworzenie
kopii tabel

Tworzy tymczasową tabelę o nazwie #Dept10 w tempdb.

Wcześniej bada czy istnieje taka tabela.

Jeśli istnieje usuwa ją poleceniem DROP TABLE

```
IF OBJECT_ID('#Dept_10') IS NOT NULL  
DROP TABLE #Dept_10;  
GO
```

```
SELECT *  
INTO #Dept_10  
FROM Employees  
WHERE department_id = 10
```

```
select * from #Dept_10
```

	employee_id	first_name	last_name	email	phone_number	hire_date	job_id	salary	commission_pct	manager_id	department_id
1	200	Jennifer	Whalen	JWHALEN	1.515.555.0165	2013-09-17	AD_ASST	4400.00	NULL	101	10